

Memory-Efficient Large Model Training: From ZeRO to ZeRO-Infinity

A Comparative Literature Review

by

Ge, Wentao

Student ID: 119010080

Final Report Submitted to
CSC6032 Advanced Operating System

The Chinese University of Hong Kong
May 2026

Abstract

Deep learning models have grown over 1000x in size in recent years, from hundreds of millions to hundreds of billions of parameters. GPU memory, however, has only increased by about 5x. This gap makes memory optimization a key challenge for large-scale training. Traditional approaches each have clear drawbacks: data parallelism copies all model states to every device and runs out of memory quickly; model parallelism and pipeline parallelism need heavy code changes and lose efficiency at scale.

This report studies the ZeRO (Zero Redundancy Optimizer) family, a line of work that pushes the boundary of trainable model size step by step. It covers three closely related systems: ZeRO, ZeRO-Offload, and ZeRO-Infinity. ZeRO partitions optimizer states, gradients, and parameters across GPUs instead of copying them. This cuts memory use in proportion to the number of devices, while keeping the simplicity and low communication cost of data parallelism. In practice, ZeRO achieves super-linear speedup on up to 400 GPUs and can train models with over 100 billion parameters. ZeRO-Offload builds on this idea by moving gradients and optimizer states to CPU memory. It also runs the parameter update step on the CPU. This strategy minimises data movement between GPU and CPU while saving the most GPU memory possible. As a result, it supports 10x larger models on a single GPU without any changes to model code. ZeRO-Infinity goes further by adding NVMe SSDs to the memory

hierarchy. With its infinity offload engine, bandwidth-centric partitioning, and memory-centric tiling, it can handle models with tens of trillions of parameters and allows fine-tuning of trillion-parameter models on a single DGX-2 node.

This report also provides a comparative analysis of the three systems. It examines memory efficiency, offloading design, communication cost, scalability, and ease of use. The analysis shows a clear progression: ZeRO partitions states across GPUs only; ZeRO-Offload extends this to GPU–CPU cooperation; ZeRO-Infinity further expands to a full GPU–CPU–NVMe hierarchy. Across all three systems, two design goals remain consistent: keeping training fast through overlapping communication with computation, and keeping the system easy to use through a data-parallelism-based interface. Finally, this report discusses open challenges and future directions, including bandwidth limits in heterogeneous offloading, the tension between small batch sizes and model convergence, and opportunities for hardware–software co-design in next-generation training systems.

Contents

Abstract

List of Figures **v**

List of Tables **vii**

1 Introduction **1**

1.1 Research Background 1

1.2 Core Problem 3

1.3 Contributions 4

1.4 Structure of the Report 4

2 Background and Related Work **6**

2.1 Memory Consumption in Model Training 6

2.2 Parallelism Strategies for Large Model Training 7

2.3 Heterogeneous Training and CPU Offloading 8

2.4 Other Memory Reduction Techniques 9

3 ZeRO: Memory Optimizations Toward Training Trillion Parameter Models **11**

3.1 Motivation 11

3.2 ZeRO-DP: Partitioning Model States 12

3.2.1	Stage 1: Optimizer State Partitioning (P_{os})	12
3.2.2	Stage 2: Gradient Partitioning (P_g)	12
3.2.3	Stage 3: Parameter Partitioning (P_p)	13
3.2.4	Communication Analysis	13
3.3	ZeRO-R: Reducing Residual Memory	14
3.4	Experimental Results	15
3.4.1	Speed and Model Scale	15
3.4.2	Super-Linear Scalability	16
3.4.3	Training Without Model Parallelism	16
3.4.4	Turing-NLG	17
3.5	Summary	17
4	ZeRO-Offload: Democratizing Billion-Scale Model Training	19
4.1	Motivation	19
4.2	Optimal Offload Strategy	20
4.2.1	Limiting CPU Computation	20
4.2.2	Minimising Communication	20
4.2.3	The Unique Optimal Strategy	21
4.3	Training Schedule and Multi-GPU Scaling	22
4.3.1	Single-GPU Schedule	22
4.3.2	Multi-GPU Scaling	22
4.4	Optimised CPU Execution	23
4.5	Experimental Results	24
4.5.1	Model Scale	24
4.5.2	Training Throughput	25
4.5.3	Scalability	25
4.6	Summary	26

5	ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning	27
5.1	Motivation	27
5.2	Design for Unprecedented Scale	28
5.2.1	Infinity Offload Engine	28
5.2.2	Memory-Centric Tiling	29
5.3	Design for Training Efficiency	29
5.3.1	Bandwidth-Centric Partitioning	29
5.3.2	Overlap-Centric Design	30
5.4	Experimental Results	30
5.4.1	Model Scale	30
5.4.2	Training Throughput	32
5.4.3	Super-Linear Scalability	34
5.4.4	Impact of System Features	34
5.5	Summary	35
6	Conclusion and Discussion	36
6.1	Summary of the Three Systems	36
6.2	Comparative Analysis	37
6.3	Shared Design Principles	38
6.4	Open Challenges and Future Directions	39
6.5	Concluding Remarks	40
	Bibliography	41

List of Figures

1.1	Growth of model size versus GPU memory capacity (2017–2021). Model parameters have increased by over 1000x while GPU memory has grown by only about 5x.	2
3.1	Per-device memory consumption of model states with three stages of ZeRO-DP optimizations. Ψ denotes model size (number of parameters), K denotes the memory multiplier of optimizer states, and N_d denotes the data-parallel degree. The example assumes $\Psi = 7.5\text{B}$ and $N_d = 64$ with $K = 12$ based on mixed-precision training with Adam.	13
3.2	Training throughput per GPU and speedup of ZeRO-100B compared to the Megatron-LM baseline for varying model sizes on 400 V100 GPUs.	16
3.3	Super-linear scalability of ZeRO-100B on a 60B parameter model from 64 to 400 GPUs. Throughput more than doubles when the number of GPUs doubles, due to reduced per-device memory enabling larger batch sizes.	17

4.1	The data-flow graph of ZeRO-Offload for a model with M parameters. The fp16 parameters and forward/backward computation stay on GPU, while gradients and fp32 optimizer states are offloaded to CPU. Communication volume is $4M$ bytes.	21
4.2	Maximum trainable model size on a single GPU, 4 GPUs, and 16 GPUs (one DGX-2 node) for PyTorch, L2L, Megatron, ZeRO-2, and ZeRO-Offload.	24
4.3	Training throughput per GPU on 16 GPUs (one DGX-2 node) comparing PyTorch, ZeRO-2, Megatron-LM, ZeRO-Offload without model parallelism, and ZeRO-Offload with model parallelism.	25
5.1	Architecture overview of ZeRO-Infinity training a model on four data-parallel ranks. Partitioned model states can reside in GPU, CPU, or NVMe memory. The infinity offload engine coordinates data movement across the full memory hierarchy.	31
5.2	ZeRO-Infinity trains models up to 20 trillion parameters on 512 GPUs, achieving up to 49 TFlops/GPU for models up to 10T. 3D parallelism is limited to about 650B parameters on the same hardware.	32
5.3	Super-linear scalability of ZeRO-Infinity from 64 to 512 GPUs for a 1 trillion parameter model. Scaling increases aggregate PCIe and NVMe bandwidth, improving offloading efficiency.	33
5.4	ZeRO-Infinity trains models up to 1 trillion parameters on a single DGX-2 node (16 GPUs) without model parallelism, achieving over 40 TFlops/GPU for models up to 100B.	33

List of Tables

3.1	Per-device memory consumption (GB) of model states under different ZeRO-DP stages as a function of data-parallel degree N_d . Bold entries indicate configurations that fit on 32 GB V100 GPUs.	15
4.1	Adam optimizer latency (seconds) comparing the optimised CPU-Adam with PyTorch CPU and PyTorch GPU implementations.	23
5.1	Maximum trainable model size on a single DGX-2 node (16 V100 GPUs) with different memory strategies.	34
6.1	Comparative analysis of ZeRO, ZeRO-Offload, and ZeRO-Infinity.	37

Chapter 1

Introduction

1.1 Research Background

Deep learning has achieved major breakthroughs in recent years, largely driven by the growth of model size. In natural language processing (NLP), the Transformer architecture [27] opened the door to large-scale pre-trained models. Starting from ELMo with around 100 million parameters [15], model sizes grew rapidly: BERT reached 340 million [5], GPT-2 reached 1.5 billion [17], and Megatron-LM pushed to 8.3 billion [26]. By 2020, GPT-3 reached 175 billion parameters [2]. Research on scaling laws has shown that larger models are not only more accurate but also more sample-efficient, meaning they need less data per parameter to reach a given quality level [10]. This trend strongly motivates continued growth in model scale.

However, hardware capacity has not kept up. As shown in Figure 1.1, over the same period that model sizes grew by over 1000x, GPU memory only increased by about 5x, from 16 GB to 80 GB [19]. Training a model with mixed precision and the Adam optimiser requires roughly $16M$ bytes of memory for model states alone (where M is the number of parameters), which far exceeds the capacity of

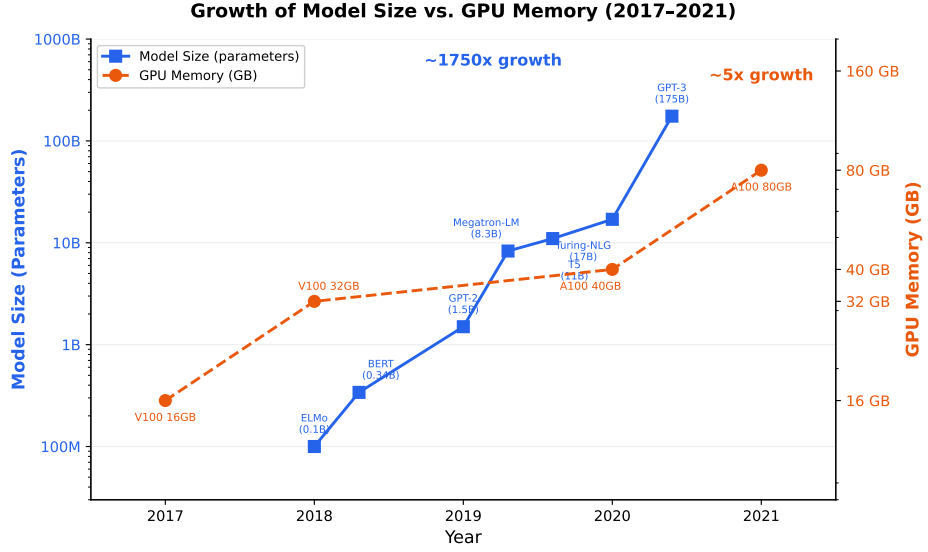


Figure 1.1: Growth of model size versus GPU memory capacity (2017–2021). Model parameters have increased by over 1000x while GPU memory has grown by only about 5x.

any single GPU for models beyond a few billion parameters [18]. This gap has made memory optimisation one of the most pressing challenges in distributed training.

The community has developed several parallelism strategies to address this challenge, including data parallelism [4, 12], model parallelism [24, 26], and pipeline parallelism [6, 8]. Each offers partial solutions but comes with significant limitations in memory efficiency, communication overhead, or usability. Other techniques such as activation checkpointing [3] and mixed precision training [13] help reduce memory pressure but cannot close the gap on their own. A detailed discussion of these approaches is provided in Chapter 2.

Against this background, the ZeRO (Zero Redundancy Optimiser) family of techniques [18, 19, 21] was developed to address the memory challenge in a more systematic way. Rather than replacing data parallelism, ZeRO builds on top

of it by removing the redundant copies of model states that data parallelism normally maintains. This line of work has progressively expanded the set of memory resources that can be used for training, from GPU memory alone, to GPU plus CPU memory, and finally to GPU plus CPU plus NVMe storage, while keeping the programming model simple and the training throughput high.

1.2 Core Problem

The core problem studied in this report is:

How can we train extremely large deep learning models efficiently without being limited by GPU memory or requiring complex model refactoring?

This question is not only about fitting a model into memory. It is about the design of a training system that must decide how to store model states across devices, how to move data between different levels of the memory hierarchy, and how to keep GPUs busy while doing so. As discussed in Chapter 2, data parallelism wastes memory on redundant copies, model and pipeline parallelism trade usability and efficiency for memory savings, and even their combination (3D parallelism) is ultimately bounded by aggregate GPU memory. At the same time, much larger and cheaper memory resources such as CPU DRAM and NVMe SSDs remain unused because existing systems cannot exploit them without being bottlenecked by their lower bandwidth. The three selected works provide three increasingly ambitious answers to this problem: ZeRO removes memory redundancy across GPUs by partitioning model states instead of replicating them, ZeRO-Offload extends the usable memory from GPU to GPU plus CPU, and ZeRO-Infinity further incorporates NVMe storage to support models with tens of trillions of parameters.

1.3 Contributions

This report provides a unified review of the ZeRO family of memory optimisation techniques, covering ZeRO [18], ZeRO-Offload [21], and ZeRO-Infinity [19], and explains how the three systems form a coherent progression from GPU-only state partitioning to full heterogeneous memory utilisation across GPU, CPU, and NVMe devices. Beyond summarising each system individually, this report presents a comparative analysis across multiple dimensions, including memory efficiency, communication overhead, offloading granularity, scalability, and ease of use, highlighting the design trade-offs made at each stage. It also identifies the shared design principles that run through the entire ZeRO family, namely communication-computation overlap for maintaining high training throughput, and a data-parallelism-compatible programming interface for keeping the engineering cost low, and discusses how these principles are preserved as the system grows more complex. Finally, this report examines open challenges and future research directions.

1.4 Structure of the Report

The remainder of this report is organised as follows. Chapter 2 provides the background and related work, covering the memory consumption breakdown during training, the key parallelism strategies (data, model, and pipeline parallelism), and earlier heterogeneous training approaches that motivate the ZeRO family. Chapter 3 presents ZeRO, which removes memory redundancy in data parallelism through three progressive stages of model state partitioning. Chapter 4 describes ZeRO-Offload, which extends the memory boundary from GPU to CPU by offloading gradients and optimiser states, enabling billion-scale training on a single GPU. Chapter 5 covers ZeRO-Infinity, which further incorporates NVMe storage

into the memory hierarchy and introduces the infinity offload engine, bandwidth-centric partitioning, and memory-centric tiling to support models with tens of trillions of parameters. Chapter 6 concludes the report with a comparative discussion of the three systems, a summary of their shared design principles, and an outline of open challenges and future research directions.

Chapter 2

Background and Related Work

This chapter provides the technical background needed to understand the ZeRO family of systems. It covers the memory consumption breakdown during training, the main parallelism strategies used for large models, heterogeneous offloading approaches, and other memory reduction techniques.

2.1 Memory Consumption in Model Training

The memory consumed during deep learning training can be divided into two categories: model states and residual states [18].

Model states include three components: parameters, gradients, and optimiser states. For a model with M parameters trained using mixed precision [13] with the Adam optimiser [11], the memory breakdown is as follows. The forward and backward passes use fp16 (2 bytes each) for both parameters and gradients, requiring $2M + 2M = 4M$ bytes. The Adam optimiser maintains an fp32 copy of the parameters, plus fp32 momentum and variance, requiring $4M + 4M + 4M = 12M$ bytes. In total, model states consume $16M$ bytes [18, 21]. For a model like GPT-2 with 1.5 billion parameters, this amounts to 24 GB, far more than the

3 GB needed for the fp16 parameters alone.

Residual states include activations, temporary buffers, and fragmented memory. Activations are the intermediate results stored during the forward pass for use in the backward pass. For a 1.5-billion-parameter GPT-2 model with sequence length 1024 and batch size 32, activations alone require about 60 GB [18]. Temporary buffers, such as those used for fusing gradients before an all-reduce operation, can also be significant: a flattened fp32 buffer for a 1.5B model requires 6 GB. Memory fragmentation further reduces usable memory, as non-contiguous free blocks cannot satisfy large allocation requests even when total free memory is sufficient.

2.2 Parallelism Strategies for Large Model Training

Data parallelism (DP) is the most widely used strategy for scaling training to multiple devices [4, 12]. In DP, each GPU holds a complete replica of the model. At each training step, the input mini-batch is split across all devices, each device performs forward and backward passes independently, and the gradients are averaged across devices using an all-reduce collective. Each device then updates its local copy of the model. DP is simple to use and has good communication efficiency: the all-reduce volume is $2M$ bytes (independent of the number of devices) when using a ring-based implementation. However, DP provides no memory savings because every device stores the full $16M$ bytes of model states. On a 32 GB V100 GPU, this limits DP to models of about 1.4 billion parameters [18].

Model parallelism (MP) splits the model vertically, distributing the parameters and computation of each layer across multiple devices [24, 26]. For example, Megatron-LM [26] partitions the weight matrices of each Transformer layer across

GPUs. MP reduces per-device memory proportionally to the number of partitions, but it requires all-reduce communication at every layer boundary. This makes MP efficient only within a single node where GPUs are connected by high-bandwidth links such as NVLink. Across nodes, efficiency drops sharply: a 40B parameter model using Megatron-LM across two DGX-2 nodes achieves only about 5 TFlops per V100 GPU, less than 5% of hardware peak [18]. MP also requires users to rewrite their model code to replace standard operators with tensor-sliced versions.

Pipeline parallelism (PP) splits the model horizontally by assigning consecutive groups of layers to different devices [6, 8, 14]. GPipe [8] divides the mini-batch into micro-batches to fill the pipeline and reduce idle time (pipeline bubbles), but requires a large batch size to keep the bubble overhead small. PipeDream [6, 14] uses asynchronous weight updates to reduce bubbles without increasing batch size, but introduces weight staleness. PP reduces per-device memory for parameters and optimiser states, but it introduces pipeline bubbles, requires careful load balancing, and demands model refactoring. Features like tied weights and batch normalisation are difficult to implement under horizontal splitting [18].

In practice, training the largest models often requires combining all three forms of parallelism (3D parallelism: DP + MP + PP), at the cost of significant system complexity and engineering effort [19].

2.3 Heterogeneous Training and CPU Offloading

Modern compute nodes contain not only GPU memory but also much larger CPU DRAM (often hundreds of gigabytes) and NVMe storage (often terabytes). Heterogeneous training approaches attempt to use these additional memory re-

sources to support larger models. Early work such as vDNN [22] virtualises GPU memory by swapping activations to CPU memory during training. SwapAdvisor [7] uses integer linear programming to decide which tensors to swap and when, reducing the performance overhead of naive swapping. Sentinel [20] focuses on efficient tensor migration between GPU and CPU memory on heterogeneous systems. L2L [16] manages memory layer by layer, synchronously moving tensors to GPU only when needed for the current layer’s computation.

These approaches share a common limitation: they introduce significant communication overhead due to the limited PCIe bandwidth between GPU and CPU (typically 16–32 GB/s per direction), and they often require model-specific refactoring or do not provide a clear path to multi-GPU scaling [21]. The ZeRO-Offload and ZeRO-Infinity systems address these limitations through careful co-design of offloading strategy with data-parallel partitioning, as discussed in later chapters.

2.4 Other Memory Reduction Techniques

Several other techniques help reduce memory pressure during training. Activation checkpointing [3] trades compute for memory by saving only a subset of activations during the forward pass and recomputing the rest during the backward pass. This reduces activation memory from $O(N)$ to approximately $O(\sqrt{N})$ at the cost of about 33% additional computation [3]. Other approaches include activation compression [9] and live analysis techniques that identify which activations can be safely discarded. These methods are complementary to the ZeRO family: ZeRO-R includes activation partitioning to remove redundant copies in model-parallel settings, and both ZeRO-Offload and ZeRO-Infinity work together with activation checkpointing [18, 19].

Mixed precision training [13] uses fp16 for most computations while maintaining an fp32 master copy of parameters for numerical stability. This halves the memory for activations and enables the use of tensor cores on modern NVIDIA GPUs. However, the fp32 master copy and optimiser states still dominate total memory consumption. Adaptive optimisers such as Adam [11] maintain per-parameter momentum and variance, requiring $12M$ bytes of optimiser states alone. Some work has explored reducing this cost: Adafactor [25] maintains factored second-order statistics, and SM3 [1] uses structured approximations. However, these memory-efficient optimisers may affect convergence and are not always drop-in replacements for Adam. The ZeRO approach is orthogonal: it reduces the per-device memory footprint of any optimiser by partitioning its states across devices, without changing the optimisation algorithm or affecting convergence [18].

Chapter 3

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models

This chapter presents ZeRO (Zero Redundancy Optimizer) [18], which eliminates memory redundancy in data-parallel training by partitioning model states across devices rather than replicating them.

3.1 Motivation

As discussed in Chapter 2, data parallelism replicates all model states on every GPU, wasting memory on redundant copies. Model parallelism reduces memory but sacrifices communication efficiency and requires code refactoring. ZeRO is motivated by three observations. First, data parallelism has better scaling efficiency than model parallelism because it maintains higher computational granularity and lower communication volume. Second, data parallelism is memory-inefficient because model states are stored redundantly across all processes. Third,

not all model states are needed at all times during training: parameters for a given layer are only required during the forward and backward passes of that layer. Based on these observations, ZeRO aims to achieve the memory efficiency of model parallelism while retaining the computational efficiency and ease of use of data parallelism.

3.2 ZeRO-DP: Partitioning Model States

ZeRO-DP removes memory redundancy by partitioning model states across data-parallel processes. It consists of three progressive stages, each partitioning an additional component of the model states.

3.2.1 Stage 1: Optimizer State Partitioning (P_{os})

In the first stage, the optimizer states are divided into N_d equal partitions (where N_d is the data-parallel degree). Each process stores and updates only its assigned $1/N_d$ fraction of the optimizer states. At the end of each training step, an all-gather operation collects the updated parameters from all processes. This reduces the memory for optimizer states from $K\Psi$ to $K\Psi/N_d$ (where K is the optimizer memory multiplier and Ψ is the number of parameters). For mixed-precision Adam ($K = 12$), the total model-state memory drops from 16Ψ to approximately 4Ψ when N_d is large, a 4x reduction.

3.2.2 Stage 2: Gradient Partitioning (P_g)

Since each process only updates its own partition of parameters, it only needs the corresponding gradients. ZeRO replaces the standard all-reduce with a reduce-scatter operation: during the backward pass, each gradient is reduced only to the process responsible for that partition. Other processes immediately free the

gradient memory. This reduces gradient memory from 2Ψ to $2\Psi/N_d$, achieving up to 8x total memory reduction when combined with P_{os} .

3.2.3 Stage 3: Parameter Partitioning (P_p)

In the final stage, each process stores only its partition of the parameters. When parameters from other partitions are needed for forward or backward computation, they are fetched on demand via an all-gather from the owning processes and discarded immediately after use. This reduces total model-state memory from 16Ψ to $16\Psi/N_d$, a linear reduction proportional to the number of devices.

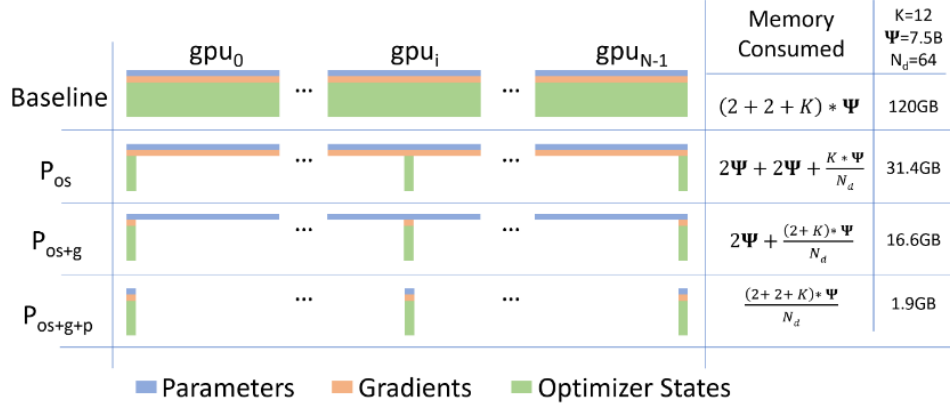


Figure 3.1: Per-device memory consumption of model states with three stages of ZeRO-DP optimizations. Ψ denotes model size (number of parameters), K denotes the memory multiplier of optimizer states, and N_d denotes the data-parallel degree. The example assumes $\Psi = 7.5\text{B}$ and $N_d = 64$ with $K = 12$ based on mixed-precision training with Adam.

3.2.4 Communication Analysis

A key property of ZeRO-DP is that it achieves large memory savings with minimal communication overhead. Stages 1 and 2 ($P_{os} + P_g$) incur the same total

communication volume as standard data parallelism (2Ψ bytes per step), because the reduce-scatter plus all-gather is equivalent to an all-reduce. Stage 3 (P_p) adds one additional all-gather per forward pass and one per backward pass for parameter fetching, increasing the total communication to 3Ψ , which is 1.5x the baseline. This modest increase enables memory reduction proportional to N_d .

3.3 ZeRO-R: Reducing Residual Memory

Beyond model states, ZeRO also addresses residual memory consumption through three techniques collectively called ZeRO-R.

Partitioned activation checkpointing (P_a) removes the redundant activation copies that model parallelism introduces. Instead of replicating activation checkpoints across all model-parallel GPUs, ZeRO partitions them and uses an all-gather to reconstruct them on demand during the backward pass. This reduces activation memory by a factor equal to the model-parallel degree. For very large models, these partitioned checkpoints can further be offloaded to CPU memory (P_{a+cpu}), reducing activation memory to nearly zero.

Constant-size buffers (C_B) address the problem of temporary buffers that grow with model size. Operations like fused all-reduce allocate buffers proportional to the total number of parameters, which becomes prohibitive for large models. ZeRO uses fixed-size buffers that are large enough to maintain high efficiency but do not grow unboundedly.

Memory defragmentation (M_D) pre-allocates contiguous memory chunks for activation checkpoints and gradients, copying them into these chunks as they are produced. This prevents out-of-memory failures caused by fragmentation and improves allocator performance.

3.4 Experimental Results

The authors implement ZeRO-100B, which includes $P_{os} + P_g$ (Stages 1 and 2) plus ZeRO-R, and evaluate it on a cluster of 400 NVIDIA V100 GPUs (25 DGX-2 nodes) with 800 Gbps inter-node bandwidth. The models are GPT-2-style Transformers with varying sizes. Table 3.1 shows the per-device memory consumption under different ZeRO-DP stages and data-parallel degrees, demonstrating how memory scales down with increasing N_d .

Table 3.1: Per-device memory consumption (GB) of model states under different ZeRO-DP stages as a function of data-parallel degree N_d . Bold entries indicate configurations that fit on 32 GB V100 GPUs.

N_d	7.5B Model			128B Model		
	P_{os}	P_{os+g}	P_{os+g+p}	P_{os}	P_{os+g}	P_{os+g+p}
1	120	120	120	2048	2048	2048
16	35.6	21.6	7.5	608	368	128
64	31.4	16.6	1.9	536	284	32
256	30.4	15.4	0.5	518	263	8
1024	30.1	15.1	0.1	513	257	2

3.4.1 Speed and Model Scale

ZeRO-100B trains models with up to 170 billion parameters, an 8x increase over the Megatron-LM baseline which cannot scale efficiently beyond 40B. ZeRO achieves a sustained throughput of 15 PetaFlops (over 38 TFlops per GPU) for models between 8B and 100B parameters. In comparison, the baseline performance degrades sharply for large models because model parallelism across nodes is bottlenecked by the inter-node bandwidth (12.5 GB/s InfiniBand vs. 300 GB/s

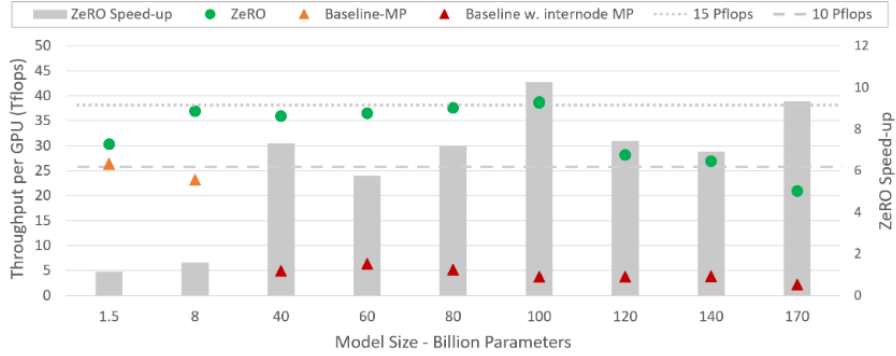


Figure 3.2: Training throughput per GPU and speedup of ZeRO-100B compared to the Megatron-LM baseline for varying model sizes on 400 V100 GPUs.

NVSwitch within a node). ZeRO achieves up to 10x speedup over the baseline for the same model size.

3.4.2 Super-Linear Scalability

ZeRO-100B demonstrates super-linear speedup when scaling from 64 to 400 GPUs on a 60B parameter model. This occurs because increasing the data-parallel degree reduces per-device memory consumption, which allows larger batch sizes per GPU, which in turn improves arithmetic intensity and throughput.

3.4.3 Training Without Model Parallelism

ZeRO-100B can train models with up to 13 billion parameters using only data parallelism (no model parallelism) on 128 GPUs, achieving over 40 TFlops per GPU. Without ZeRO, the largest model trainable with data parallelism alone is 1.4B parameters. This eliminates the need for model refactoring and makes large-model training accessible to researchers who do not have expertise in model parallelism.

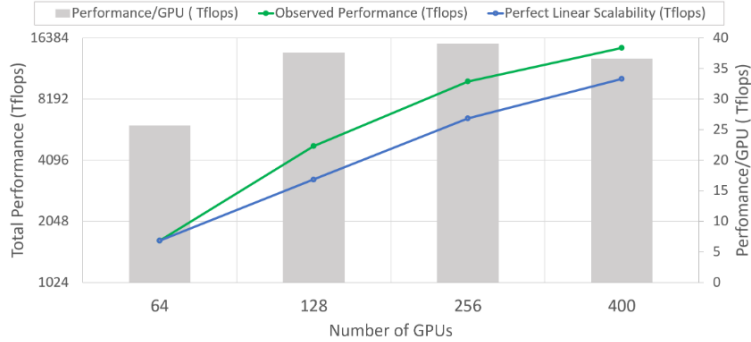


Figure 3.3: Super-linear scalability of ZeRO-100B on a 60B parameter model from 64 to 400 GPUs. Throughput more than doubles when the number of GPUs doubles, due to reduced per-device memory enabling larger batch sizes.

3.4.4 Turing-NLG

ZeRO-100B was used to train Turing-NLG [23], a 17-billion-parameter language model that achieved state-of-the-art accuracy on the Webtext-103 benchmark (perplexity of 10.21). Training sustained 41.4 TFlops per GPU throughout.

3.5 Summary

ZeRO introduces a partitioning-based approach to data-parallel training that removes memory redundancy without sacrificing efficiency or usability. Its three stages of model-state partitioning reduce per-device memory by up to N_d times (linear in the number of devices), while increasing communication by at most 1.5x. ZeRO-R further addresses residual memory through activation partitioning, constant-size buffers, and memory defragmentation. The implementation demonstrates 8x larger models, 10x faster training, and super-linear scalability compared to the state of the art. Importantly, ZeRO requires no model code changes and is as easy to use as standard data parallelism. However, ZeRO’s

memory savings are still bounded by the aggregate GPU memory of the cluster, which motivates the heterogeneous offloading approaches discussed in the following chapters.

Chapter 4

ZeRO-Offload: Democratizing Billion-Scale Model Training

This chapter presents ZeRO-Offload [21], which extends the ZeRO partitioning strategy into a heterogeneous GPU–CPU architecture, enabling billion-scale model training on a single GPU without sacrificing efficiency or requiring model code changes.

4.1 Motivation

While ZeRO (Chapter 3) removes memory redundancy across GPUs, its memory savings are still bounded by the aggregate GPU memory of the cluster. For researchers with limited hardware, such as a single GPU or a small number of GPUs, ZeRO alone cannot support models beyond a few billion parameters. At the same time, every machine has CPU memory that is significantly larger and cheaper than GPU memory (hundreds of gigabytes vs. tens of gigabytes), yet this resource remains unused during standard training.

The key question ZeRO-Offload addresses is: can we train 10-billion-parameter

models on a single GPU by leveraging CPU memory, without making training unacceptably slow? Naive offloading approaches suffer from the limited PCIe bandwidth between GPU and CPU (16–32 GB/s), which can easily become a bottleneck. ZeRO-Offload solves this through a principled offload strategy that provably minimises communication while maximising memory savings.

4.2 Optimal Offload Strategy

ZeRO-Offload models the training process as a data-flow graph and uses first-principle analysis to find the optimal partitioning between GPU and CPU. The strategy is designed around three goals: (1) limit CPU computation to avoid it becoming a bottleneck, (2) minimise GPU–CPU communication volume, and (3) maximise GPU memory savings.

4.2.1 Limiting CPU Computation

The compute complexity of forward and backward propagation is $O(MB)$ (where M is model size and B is batch size), while the parameter update has complexity $O(M)$. Since CPU throughput is orders of magnitude lower than GPU, only $O(M)$ computations (parameter updates, norm calculations) should be offloaded to CPU. The forward and backward passes must remain on GPU. ZeRO-Offload fuses these into a single “FWD-BWD super-node” assigned to the GPU.

4.2.2 Minimising Communication

The minimum communication volume for any offload strategy is $4M$ bytes (two edges of $2M$ each must be cut in the data-flow graph cycle). ZeRO-Offload achieves this minimum by co-locating the fp32 optimizer states with their producer and consumer operations on the CPU. Any strategy that separates them

would require at least $6M$ bytes of communication.

4.2.3 The Unique Optimal Strategy

The analysis yields a unique solution: GPU stores the fp16 parameters and performs forward and backward computation; CPU stores the fp32 optimizer states (parameters, momentum, variance) and the fp16 gradients, and performs the parameter update. This is the only strategy that simultaneously achieves minimum communication ($4M$), minimum CPU computation ($O(M)$), and maximum GPU memory savings.

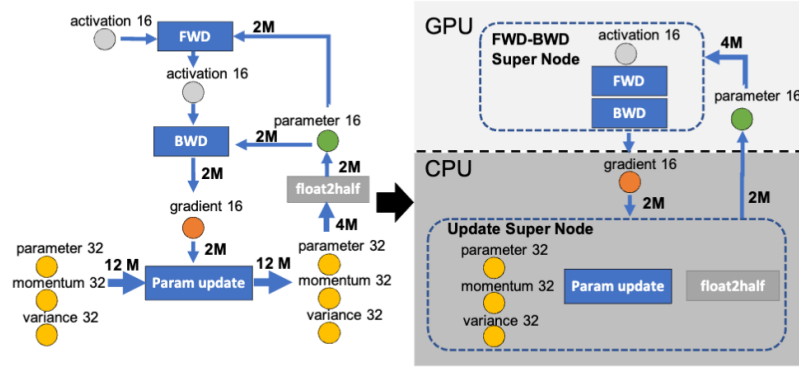


Figure 4.1: The data-flow graph of ZeRO-Offload for a model with M parameters. The fp16 parameters and forward/backward computation stay on GPU, while gradients and fp32 optimizer states are offloaded to CPU. Communication volume is $4M$ bytes.

4.3 Training Schedule and Multi-GPU Scaling

4.3.1 Single-GPU Schedule

On a single GPU, the training proceeds as follows. The forward pass runs on GPU using the fp16 parameters already in GPU memory. During the backward pass, gradients are transferred to CPU memory immediately after computation, overlapping the transfer with the remaining backward computation. After the backward pass completes, the CPU updates the fp32 parameters and optimizer states using its optimised Adam implementation, then copies the updated fp16 parameters back to GPU for the next step. The total GPU–CPU communication per step is $4M$ bytes: $2M$ for gradients (GPU $\rightarrow$ CPU) and $2M$ for updated parameters (CPU $\rightarrow$ GPU).

4.3.2 Multi-GPU Scaling

ZeRO-Offload integrates with ZeRO Stage 2 (optimizer state and gradient partitioning) to scale across multiple GPUs. Each GPU computes forward and backward passes on the full model, but only holds its partition of gradients and optimizer states on its associated CPU. After the backward pass, a reduce-scatter operation distributes gradients to the responsible GPU, which then offloads them to its CPU for the parameter update. An all-gather collects the updated parameters across GPUs. This design achieves near-linear scalability because the per-GPU communication volume to CPU remains constant regardless of the number of GPUs.

ZeRO-Offload can also be combined with model parallelism to support even larger models. On a single DGX-2 node with 16 GPUs, combining ZeRO-Offload with model parallelism enables training of models up to 70 billion parameters.

4.4 Optimised CPU Execution

To prevent CPU computation from becoming a bottleneck, ZeRO-Offload introduces two optimisations.

The first is a high-performance CPU Adam implementation that exploits SIMD vector instructions (AVX-512), loop unrolling, and OpenMP multi-threading. As shown in Table 4.1, this achieves over 6x speedup compared to the PyTorch CPU Adam implementation. Additionally, the parameter copy from CPU to GPU is performed in a tiled manner, overlapping Adam computation on the current tile with the GPU transfer of the previous tile.

Table 4.1: Adam optimizer latency (seconds) comparing the optimised CPU-Adam with PyTorch CPU and PyTorch GPU implementations.

# Parameters	CPU-Adam	PyTorch CPU	PyTorch GPU
1B	0.22	1.39	0.10
2B	0.51	2.75	0.26
4B	1.03	5.71	0.64
8B	2.41	11.93	0.87
10B	2.57	14.76	1.00

The second is one-step delayed parameter update (DPU), which overlaps the CPU optimizer step with the GPU forward and backward computation of the next training step. DPU is activated after a few dozen warm-up iterations to avoid destabilising early training. Experiments show that DPU does not affect final model accuracy while providing 1.12–1.59x throughput improvement for small batch sizes.

4.5 Experimental Results

ZeRO-Offload is evaluated on a DGX-2 node (16 NVIDIA V100 GPUs with 32 GB each) using GPT-2-style Transformer models of varying sizes.

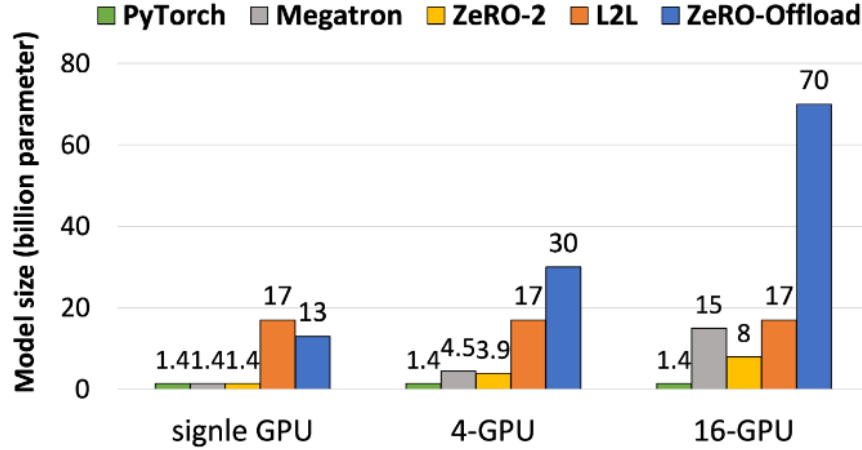


Figure 4.2: Maximum trainable model size on a single GPU, 4 GPUs, and 16 GPUs (one DGX-2 node) for PyTorch, L2L, Megatron, ZeRO-2, and ZeRO-Offload.

4.5.1 Model Scale

On a single GPU, ZeRO-Offload trains models with up to 13 billion parameters, a 10x increase over PyTorch, Megatron, and ZeRO-2, all of which are limited to 1.4B on a single 32 GB GPU. On 16 GPUs within a single DGX-2 node, ZeRO-Offload without model parallelism supports up to 13B parameters, while combining it with model parallelism extends this to 70B parameters, a 4.5x increase over model parallelism alone (which supports only 15B).

4.5.2 Training Throughput

On a single GPU, ZeRO-Offload achieves 40 TFlops for a 10B parameter model, compared to 30 TFlops for PyTorch on a 1.4B model (the largest it can fit). ZeRO-Offload outperforms L2L by a significant margin because its communication volume ($4M$) is 7x smaller than L2L’s ($28M$).

On 16 GPUs, ZeRO-Offload achieves the highest throughput for models from 1B to 15B, with up to 1.33x speedup over PyTorch, 1.11x over ZeRO-2, and 1.64x over Megatron. The additional memory savings from offloading allow larger micro-batch sizes, which improve GPU utilisation.

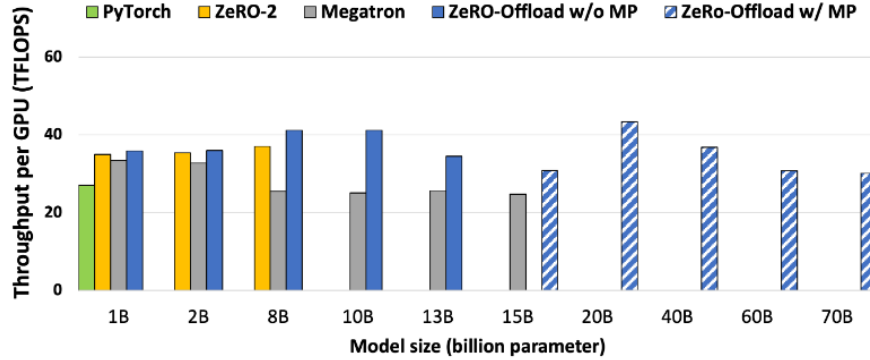


Figure 4.3: Training throughput per GPU on 16 GPUs (one DGX-2 node) comparing PyTorch, ZeRO-2, Megatron-LM, ZeRO-Offload without model parallelism, and ZeRO-Offload with model parallelism.

4.5.3 Scalability

ZeRO-Offload achieves near-linear scalability on up to 128 GPUs for a 10B parameter model. For small GPU counts (1–32), ZeRO-Offload slightly outperforms ZeRO-2 because its extra memory savings enable larger batch sizes. For larger GPU counts (64–128), ZeRO-2 begins to match or slightly exceed ZeRO-Offload

because both can run similar batch sizes, and ZeRO-2 avoids the CPU–GPU communication overhead. The two approaches are complementary: ZeRO-Offload excels when GPU memory is the bottleneck, while ZeRO-2 excels when sufficient GPUs are available.

4.6 Summary

ZeRO-Offload extends the ZeRO partitioning philosophy from GPU-only to a heterogeneous GPU–CPU architecture. Its offload strategy is provably optimal and unique: no other strategy can achieve more memory savings without increasing CPU computation or communication volume. The system enables a 10x increase in trainable model size on a single GPU, scales near-linearly to 128 GPUs, and requires no model code changes. The key limitation is that model scale remains bounded by the parameters that can fit in a single GPU’s memory (since parameters stay on GPU), and the system requires large batch sizes to amortise the CPU–GPU communication cost. These limitations motivate ZeRO-Infinity, discussed in the next chapter.

Chapter 5

ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning

This chapter presents ZeRO-Infinity [19], which extends the ZeRO family to a full GPU–CPU–NVMe memory hierarchy, enabling training of models with tens of trillions of parameters on current hardware without model code refactoring.

5.1 Motivation

ZeRO-Offload (Chapter 4) enables billion-scale training on limited GPU resources by offloading to CPU memory. However, it has two key limitations. First, model parameters must still reside in GPU memory and are replicated across devices, so the maximum model size is bounded by the memory of a single GPU. Second, it requires large batch sizes to amortise the PCIe communication cost, which limits scalability and can hurt convergence.

As models scale toward trillions of parameters, even CPU memory becomes

insufficient. A one-trillion-parameter model requires approximately 16 TB for model states, far exceeding the CPU memory of any single node. However, modern GPU clusters have massive NVMe SSD storage (often 50x larger than GPU memory and 20x larger than CPU memory per node) that remains completely unused during training. The challenge is how to exploit NVMe storage despite its limited bandwidth (1–3 GB/s per device) without making training unacceptably slow.

ZeRO-Infinity addresses this by building on ZeRO Stage 3 (full parameter partitioning) and introducing three innovations: an infinity offload engine that can place any model state on GPU, CPU, or NVMe; bandwidth-centric partitioning that achieves virtually unlimited heterogeneous memory bandwidth; and memory-centric tiling that handles extremely large individual layers.

5.2 Design for Unprecedented Scale

5.2.1 Infinity Offload Engine

ZeRO-Infinity is built on top of ZeRO-3, which partitions all model states (parameters, gradients, and optimizer states) across data-parallel processes. The infinity offload engine extends this by allowing each partitioned state to reside in GPU memory, CPU memory, or NVMe storage, depending on the available resources. Since even a 100-trillion-parameter model’s states can fit in the aggregate NVMe storage of a moderately sized cluster (e.g., 96 DGX-2 nodes), the infinity offload engine removes GPU memory as a constraint on model scale entirely.

The engine also supports CPU offloading of activation checkpoints. For models beyond 10 trillion parameters, activation checkpoints become too large for GPU memory even with checkpointing enabled. Offloading them to CPU memory (which has sufficient capacity) keeps the GPU free for computation.

5.2.2 Memory-Centric Tiling

For extremely large models, individual layers can have parameters so large that they do not fit in GPU memory even one at a time. For example, a linear layer transforming a hidden dimension of 64K to 256K requires several gigabytes of contiguous memory. Memory fragmentation makes this worse: even if total free memory is sufficient, a lack of contiguous blocks causes allocation failures.

Memory-centric tiling addresses this by breaking a single large operator into smaller tiles that are executed sequentially. Each tile’s parameters are fetched, used for a portion of the computation, and then discarded before the next tile is loaded. This allows ZeRO-Infinity to train models with hidden dimensions up to 64K without requiring model parallelism, compared to a maximum of 8K without tiling.

5.3 Design for Training Efficiency

5.3.1 Bandwidth-Centric Partitioning

A key insight of ZeRO-Infinity is that the way parameters are partitioned across processes determines the achievable bandwidth from slow memory. In ZeRO-3 and ZeRO-Offload, each layer’s parameters are owned by a single process, which must first move them from CPU or NVMe to its own GPU before broadcasting to others. This means only one PCIe link is active during the initial transfer, limiting bandwidth to about 12 GB/s regardless of the number of GPUs.

ZeRO-Infinity instead partitions each individual parameter across all data-parallel processes and uses an all-gather (rather than broadcast) to reconstruct it. This activates all PCIe links in parallel: each GPU fetches its $1/N_d$ portion from its local NVMe/CPU simultaneously. The effective bandwidth scales linearly

with the data-parallel degree. On a single DGX-2 node (16 GPUs), this achieves approximately 48 GB/s from CPU and 25 GB/s from NVMe. On 64 nodes, ZeRO-Infinity has access to over 3 TB/s of CPU memory bandwidth and over 1.5 TB/s of NVMe bandwidth, far exceeding what is needed for efficient training.

5.3.2 Overlap-Centric Design

Accessing NVMe memory involves a three-step pipeline: (1) read from NVMe to CPU memory, (2) copy from CPU to GPU memory, (3) execute all-gather across GPUs. ZeRO-Infinity overlaps all three stages with GPU computation through dynamic prefetching. Before a layer’s forward or backward pass begins, the system prefetches the next layer’s parameters through the NVMe→CPU→GPU pipeline while the current layer is being computed. This hides the data movement latency behind useful computation.

The overlap-centric design also applies to gradient offloading during the backward pass: gradients are transferred to their destination (CPU or NVMe) immediately after computation, overlapping with the backward pass of subsequent layers.

5.4 Experimental Results

ZeRO-Infinity is evaluated on NVIDIA V100 DGX-2 clusters with up to 512 GPUs (32 nodes). The models are Transformer-based with configurations ranging from 10B to 32T parameters.

5.4.1 Model Scale

ZeRO-Infinity trains models with over 32 trillion parameters on 512 GPUs, a 50x increase over 3D parallelism which is limited to about 650B on the same

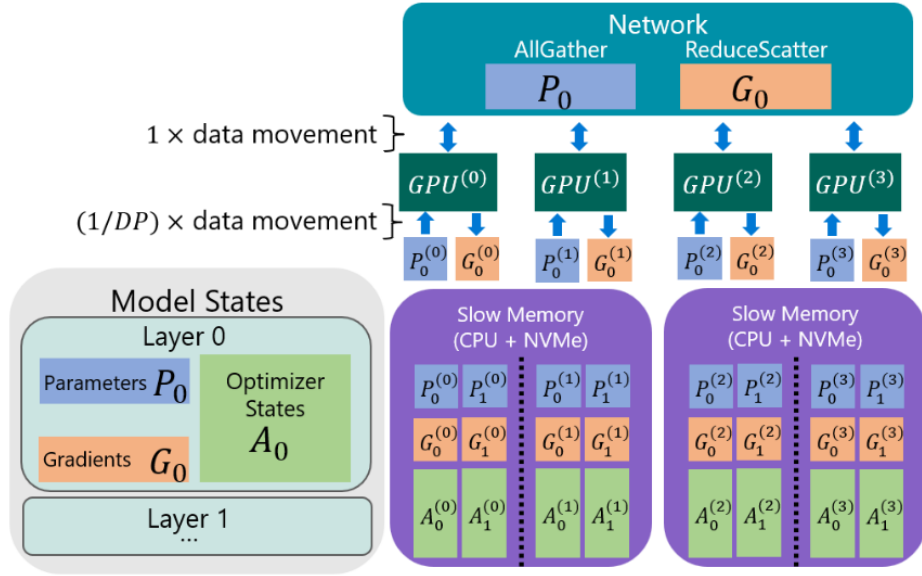


Figure 5.1: Architecture overview of ZeRO-Infinity training a model on four data-parallel ranks. Partitioned model states can reside in GPU, CPU, or NVMe memory. The infinity offload engine coordinates data movement across the full memory hierarchy.

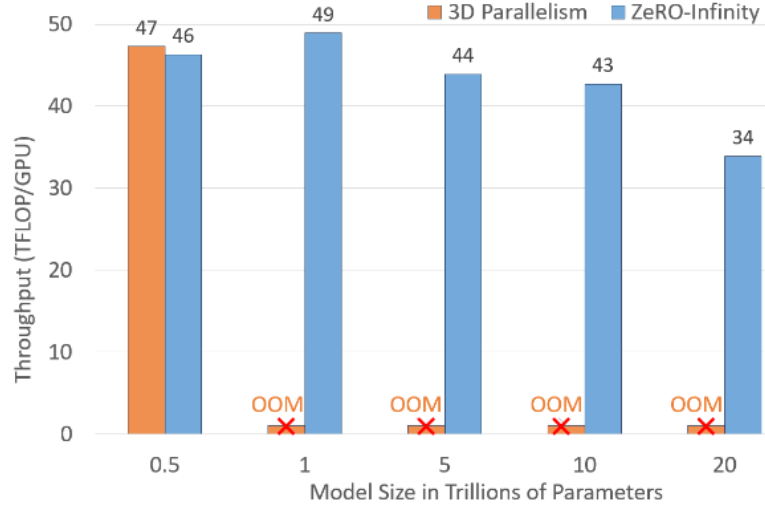


Figure 5.2: ZeRO-Infinity trains models up to 20 trillion parameters on 512 GPUs, achieving up to 49 TFlops/GPU for models up to 10T. 3D parallelism is limited to about 650B parameters on the same hardware.

hardware. On a single DGX-2 node (16 GPUs), ZeRO-Infinity trains models up to 1 trillion parameters without model parallelism, representing a 700x increase over standard data parallelism.

Table 5.1 shows how different ZeRO strategies progressively increase the maximum trainable model size on a single DGX-2 node.

5.4.2 Training Throughput

For a 500B parameter model (close to the largest 3D parallelism can handle), ZeRO-Infinity achieves nearly identical throughput to 3D parallelism, demonstrating that the offloading overhead is negligible at this scale. For larger models up to 20T parameters, ZeRO-Infinity sustains up to 49 TFlops per GPU. Performance drops slightly at 20T (34 TFlops/GPU) due to extremely small batch sizes caused by limited CPU memory for activation checkpoints, not due to NVMe

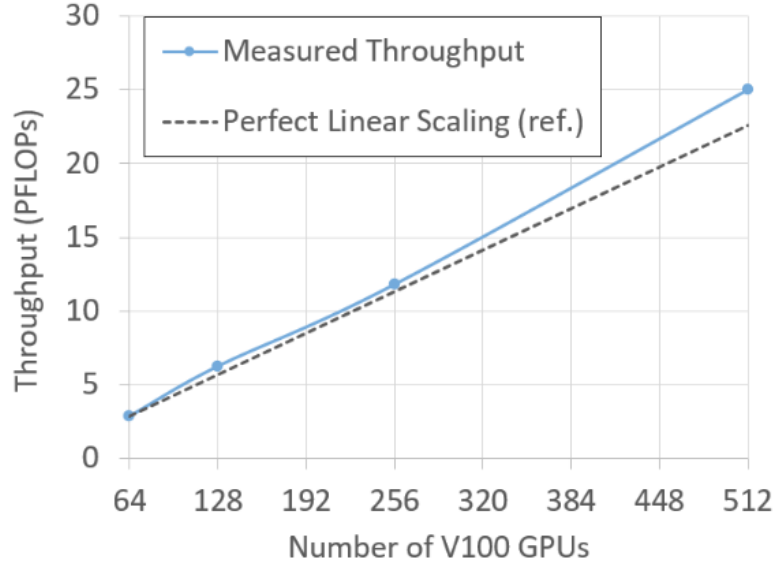


Figure 5.3: Super-linear scalability of ZeRO-Infinity from 64 to 512 GPUs for a 1 trillion parameter model. Scaling increases aggregate PCIe and NVMe bandwidth, improving offloading efficiency.

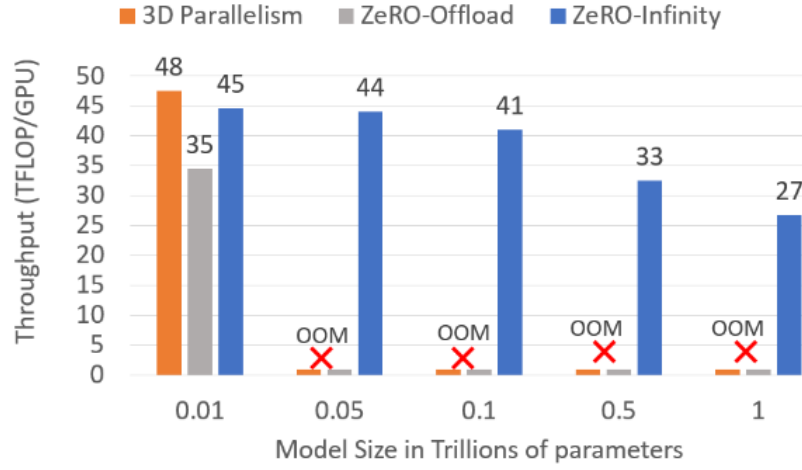


Figure 5.4: ZeRO-Infinity trains models up to 1 trillion parameters on a single DGX-2 node (16 GPUs) without model parallelism, achieving over 40 TFlops/GPU for models up to 100B.

Table 5.1: Maximum trainable model size on a single DGX-2 node (16 V100 GPUs) with different memory strategies.

Strategy	Max Model Size	Memory Used
Data Parallelism	1.4B	GPU only
ZeRO-2	13B (16 GPUs)	GPU (partitioned opt+grad)
ZeRO-Offload	13B (1 GPU)	GPU + CPU
ZeRO-Infinity (CPU)	$\sim 100\text{B}$	GPU + CPU (all states)
ZeRO-Infinity (NVMe)	1T	GPU + CPU + NVMe

bandwidth limitations.

5.4.3 Super-Linear Scalability

ZeRO-Infinity achieves super-linear scalability from 64 to 512 GPUs when training a 1 trillion parameter model. This occurs because scaling to more nodes linearly increases the aggregate PCIe and NVMe bandwidth available for offloading, as well as the aggregate CPU compute for parameter updates. Even at a modest scale of 4 nodes (64 GPUs), ZeRO-Infinity achieves over 2.8 PetaFlops (44 TFlops/GPU), demonstrating that aggregate NVMe bandwidth is sufficient for good efficiency.

5.4.4 Impact of System Features

The bandwidth-centric partitioning provides nearly 2x speedup over ZeRO-Offload’s broadcast-based approach at 64 GPUs for gradient offloading, by leveraging aggregate PCIe bandwidth. The overlap-centric design with prefetching provides up to 1.8x throughput improvement compared to a naive sequential implementation. Memory-centric tiling enables hidden dimensions up to 64K (vs. 8K without

tiling) with minimal performance overhead.

5.5 Summary

ZeRO-Infinity completes the ZeRO family’s progression from GPU-only to full heterogeneous memory utilisation. By combining ZeRO-3 partitioning with the infinity offload engine, bandwidth-centric partitioning, and memory-centric tiling, it removes GPU memory as a constraint on model scale entirely. The system trains models with tens of trillions of parameters, achieves throughput comparable to 3D parallelism, demonstrates super-linear scalability, and requires no model code changes. Its key insight is that by partitioning data across all processes before offloading, the aggregate bandwidth of all PCIe and NVMe links can be exploited in parallel, overcoming the bandwidth limitations that constrained earlier heterogeneous approaches. The remaining bottleneck is compute: training a 100-trillion-parameter model in reasonable time requires hardware with significantly more compute capacity than current GPUs, but ZeRO-Infinity ensures that memory will not be the limiting factor.

Chapter 6

Conclusion and Discussion

This chapter summarises the three systems studied in this report, provides a comparative analysis across key dimensions, identifies shared design principles, and discusses open challenges and future research directions.

6.1 Summary of the Three Systems

The ZeRO family represents a progressive line of work that expands the boundary of trainable model scale by systematically exploiting more memory resources while preserving training efficiency and ease of use.

ZeRO [18] addresses the memory redundancy in standard data parallelism by partitioning optimizer states, gradients, and parameters across GPUs. Its three stages reduce per-device memory by up to N_d times (linear in the number of devices) with at most 1.5x communication overhead. The implementation trains models up to 170B parameters on 400 GPUs with super-linear scalability and requires no model code changes.

ZeRO-Offload [21] extends the partitioning strategy to a heterogeneous GPU–CPU architecture. It offloads gradients and optimizer states to CPU memory

and runs the parameter update on CPU, following a provably optimal strategy that minimises GPU–CPU communication (4M bytes) while maximising GPU memory savings. This enables 10x larger models on a single GPU and scales near-linearly to 128 GPUs.

ZeRO-Infinity [19] further incorporates NVMe SSDs into the memory hierarchy. Through its infinity offload engine, bandwidth-centric partitioning, and memory-centric tiling, it trains models with tens of trillions of parameters and achieves super-linear scalability on up to 512 GPUs. It removes GPU memory as a constraint on model scale entirely.

6.2 Comparative Analysis

Table 6.1 provides a side-by-side comparison of the three systems across key dimensions.

Table 6.1: Comparative analysis of ZeRO, ZeRO-Offload, and ZeRO-Infinity.

	ZeRO	ZeRO-Offload	ZeRO-Infinity
Memory resources used	GPU	GPU + CPU	GPU + CPU + NVMe
Built on ZeRO stage	Stages 1–3	Stage 2	Stage 3
Max model size demonstrated	170B (400 GPUs)	70B (16 GPUs + MP)	32T (512 GPUs)
Max model on single node	13B (no MP)	13B (1 GPU)	1T (16 GPUs, no MP)
Communication overhead vs DP	1.0–1.5x	1.0x (+ 4M to CPU)	1.5x (+ offload)
Parameter placement	GPU (partitioned)	GPU (replicated)	GPU/CPU/NVMe (partitioned)
Optimizer placement	GPU (partitioned)	CPU	CPU or NVMe
Bandwidth strategy	GPU–GPU only	Single PCIe link	Aggregate all PCIe/NVMe
Requires model refactoring	No	No	No
Peak throughput reported	38+ TFlops/GPU	40 TFlops/GPU	49 TFlops/GPU

Several observations emerge from this comparison. First, each system expands the set of exploitable memory resources: ZeRO uses only GPU memory more efficiently, ZeRO-Offload adds CPU memory, and ZeRO-Infinity adds

NVMe storage. Second, the systems build on progressively deeper ZeRO stages: ZeRO-Offload uses Stage 2 (optimizer + gradient partitioning) because it still replicates parameters on GPU, while ZeRO-Infinity requires Stage 3 (full parameter partitioning) to enable parameter offloading. Third, the bandwidth strategy evolves from relying solely on GPU–GPU interconnects, to a single PCIe link per GPU, to aggregating all PCIe and NVMe links in parallel. This last innovation is what allows ZeRO-Infinity to overcome the bandwidth wall that limited earlier heterogeneous approaches.

6.3 Shared Design Principles

Despite their differences in scope and complexity, the three systems share two consistent design principles.

The first is communication-computation overlap. All three systems hide data movement latency behind useful GPU computation. ZeRO overlaps parameter all-gather with forward/backward computation. ZeRO-Offload overlaps gradient transfer to CPU with the backward pass. ZeRO-Infinity extends this to a three-stage pipeline (NVMe→CPU→GPU) with dynamic prefetching. This principle ensures that memory optimisation does not come at the cost of GPU utilisation.

The second is a data-parallelism-compatible interface. All three systems are implemented as drop-in replacements for standard data parallelism in PyTorch. Users wrap their model with the ZeRO interface and do not need to modify model code, partition layers, or manage device placement manually. This dramatically lowers the barrier to large-model training compared to model parallelism or pipeline parallelism, which require significant engineering effort.

6.4 Open Challenges and Future Directions

While the ZeRO family has made remarkable progress, several challenges remain.

Bandwidth limitations at extreme scale. ZeRO-Infinity’s bandwidth-centric partitioning scales well with the number of nodes, but on a single node the aggregate PCIe bandwidth is still limited. For very large models on small clusters, the offloading overhead can reduce throughput, particularly with small batch sizes. Future hardware with higher PCIe bandwidth (Gen 5/6) or direct GPU–NVMe links (such as GPUDirect Storage) could alleviate this.

Batch size and convergence trade-offs. Memory-constrained settings often force small per-GPU batch sizes, which reduce GPU utilisation and may require gradient accumulation. At the other extreme, scaling to many GPUs increases the effective batch size, which can slow convergence beyond the critical batch size. Finding the right balance between memory efficiency, hardware utilisation, and convergence remains an open problem.

Activation memory at extreme scale. While model states can be offloaded to NVMe, activation memory during the forward pass must reside in GPU or CPU memory for timely access during backpropagation. For models with tens of trillions of parameters, even checkpointed activations can exceed CPU memory. Offloading activations to NVMe with sufficient bandwidth is a potential direction but has not yet been demonstrated at scale.

Hardware–software co-design. The ZeRO family is designed around current hardware constraints (PCIe bandwidth, NVMe throughput, CPU compute). Future accelerator architectures with unified memory, higher-bandwidth interconnects, or compute-near-storage capabilities could enable fundamentally different system designs. Co-designing hardware and training systems together offers opportunities for the next generation of extreme-scale training.

Integration with other parallelism strategies. While ZeRO reduces the

need for model and pipeline parallelism, the largest production training runs still combine multiple forms of parallelism. How to optimally combine ZeRO-style memory optimisation with tensor parallelism, pipeline parallelism, and expert parallelism (for mixture-of-experts models) remains an active area of research.

6.5 Concluding Remarks

This report has presented a systematic study of the ZeRO family of memory optimisation techniques. The three systems form a coherent progression: ZeRO removes redundancy within GPU memory, ZeRO-Offload extends usable memory to CPU, and ZeRO-Infinity further extends it to NVMe storage. Together, they have expanded the trainable model scale from billions to tens of trillions of parameters while maintaining high training throughput and requiring no model code changes. The key lesson from this line of work is that memory efficiency and training efficiency are not fundamentally at odds: through careful partitioning, overlap, and bandwidth aggregation, it is possible to exploit the full memory hierarchy of modern clusters without sacrificing GPU utilisation. As model sizes continue to grow, the principles established by the ZeRO family will remain central to the design of next-generation training systems.

Bibliography

- [1] Rohan Anil, Vineet Gupta, Tomer Koren, and Yoram Singer. Memory-efficient adaptive optimization for large-scale learning. *ArXiv*, abs/1901.11150, 2019. [10](#)
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020. [1](#)
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *CoRR*, abs/1604.06174, 2016. [2](#), [9](#)
- [4] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’Aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc V. Le, and Andrew Y. Ng. Large scale distributed deep networks. In *Advances in Neural Information Processing Systems*, 2012. [2](#), [7](#)
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. [1](#)
- [6] Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, and Phillip B. Gibbons. PipeDream:

- Fast and efficient pipeline parallel DNN training. *CoRR*, abs/1806.03377, 2018. [2](#), [8](#)
- [7] Chien-Chin Huang, Gu Jin, and Jinyang Li. SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 1341–1355, 2020. [9](#)
- [8] Yanping Huang, Yonglong Cheng, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, 2019. [2](#), [8](#)
- [9] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *ArXiv*, abs/1910.02653, 2019. [9](#)
- [10] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. [1](#)
- [11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. [6](#), [10](#)
- [12] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and

- Soumith Chintala. PyTorch distributed: Experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, 2020. [2](#), [7](#)
- [13] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018. [2](#), [6](#), [10](#)
- [14] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Granger, Phil Gibbons, and Matei Zaharia. PipeDream: Generalized pipeline parallelism for DNN training. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2019. [8](#)
- [15] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*, 2018. [1](#)
- [16] Bharadwaj Pudipeddi, Maral Mesmakhosroshahi, Jinwen Xi, and Sujeeth Bharadwaj. Training large neural networks with constant memory using a new execution algorithm. *ArXiv*, abs/2002.05645, 2020. [9](#)
- [17] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019. OpenAI Technical Report. [1](#)
- [18] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '20. IEEE Press, 2020. [2](#), [4](#), [6](#), [7](#), [8](#), [9](#), [10](#), [11](#), [36](#)

- [19] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21. IEEE Press, 2021. [1](#), [2](#), [4](#), [8](#), [9](#), [27](#), [37](#)
- [20] Jie Ren, Jiaolin Luo, Kai Wu, Minjia Zhang, Hyeran Jeon, and Dong Li. Sentinel: Efficient tensor migration and allocation on heterogeneous memory systems for deep learning. In *International Symposium on High Performance Computer Architecture (HPCA)*, 2020. [9](#)
- [21] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. ZeRO-Offload: Democratizing billion-scale model training. In *USENIX Annual Technical Conference*, 2021. [2](#), [4](#), [6](#), [9](#), [19](#), [36](#)
- [22] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–13, 2016. [9](#)
- [23] Corby Rosset. Turing-NLG: A 17-billion-parameter language model by Microsoft, 2020. [17](#)
- [24] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, Hyoungho Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake A. Hechtman. Mesh-TensorFlow: Deep learning for supercomputers. In *Advances in Neural Information Processing Systems*, 2018. [2](#), [7](#)

- [25] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. *CoRR*, abs/1804.04235, 2018. [10](#)
- [26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019. [1](#), [2](#), [7](#)
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. [1](#)